



CENG 5410

Advanced Computer Architecture

Lecture 01: Introduction

Zhengrong Wang
CSE Department, CUHK
zhengrongwang@cuhk.edu.hk



Welcome!

- About Me:
 - Prof. @CUHK since September 2025
 - Research focuses on architectures/accelerators/specialization/heterogeneous computers
 - Love teaching this course – strong ties to research area
- Teaching Style: non-authoritarian?
 - Much of this course contains architecture “wisdom”
 - Applications and Technology change very quickly -- everything is open for debate.
- Let's treat this like a discussion!
- Slides adapted from UW Madison, UPenn, UCLA, and various universities.



Warmup 1

- Which of these is faster?

Version 1

```
void copyij(int src[2048][2048],
            int dst[2048][2048])
{
    int i,j;
    for (i = 0; i < 2048; i++)
        for (j = 0; j < 2048; j++)
            dst[i][j] = src[i][j];
}
```

Version 2

```
void copyji(int src[2048][2048],
            int dst[2048][2048])
{
    int i,j;
    for (j = 0; j < 2048; j++)
        for (i = 0; i < 2048; i++)
            dst[i][j] = src[i][j];
}
```



Warmup 2

- Which of these is faster?

Version 1

```
for (unsigned c = 0; c < n; ++c)
    data[c] = std::rand() % 256;

//std::sort(data, data + n);

// BEGIN TIMER
for (int c = 0; c < n; ++c)
    if (data[c] >= 128)
        sum += data[c]*2;
// END TIMER
```

Version 2

```
for (unsigned c = 0; c < n; ++c)
    data[c] = std::rand() % 256;

std::sort(data, data + n);

// BEGIN TIMER
for (int c = 0; c < n; ++c)
    if (data[c] >= 128)
        sum += data[c]*2;
// END TIMER
```



What is computer architecture?

- The intersection between software and hardware.
- Hardware: microarchitecture, performance, power, area.
- ISA: instruction set architecture.
- OS/Runtime/Application: software leveraging the ISA and hardware.



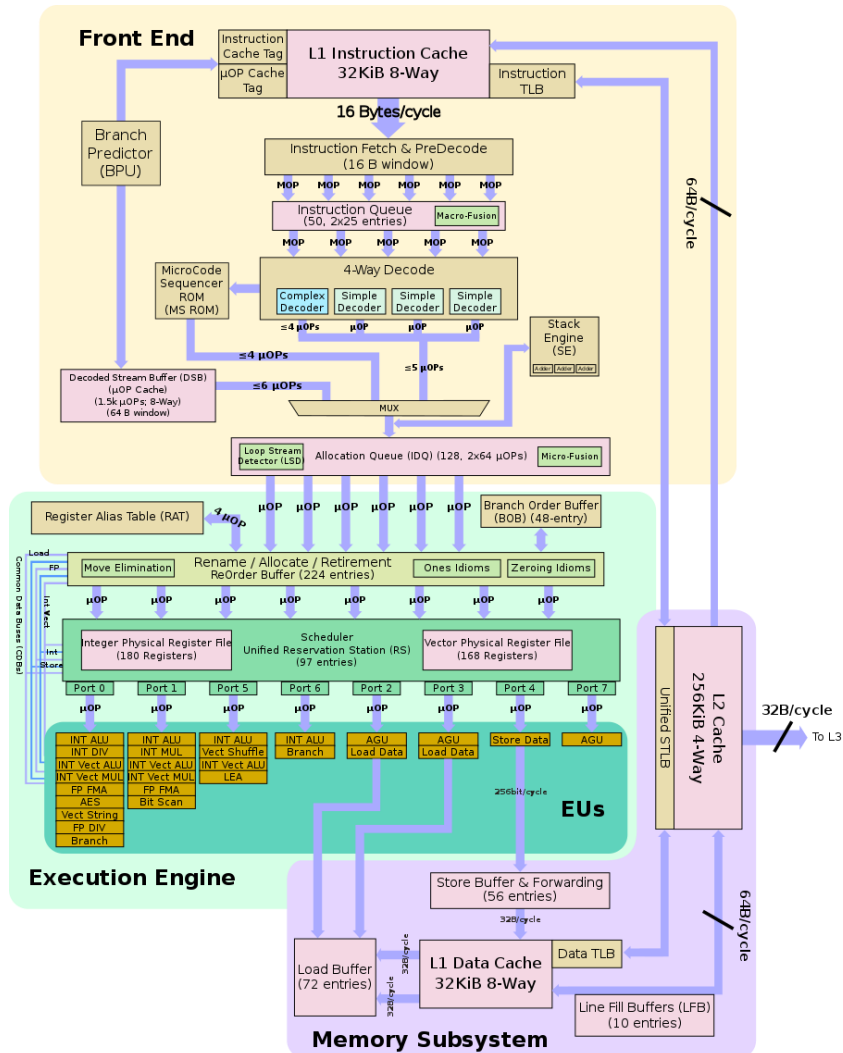
Example Architectures



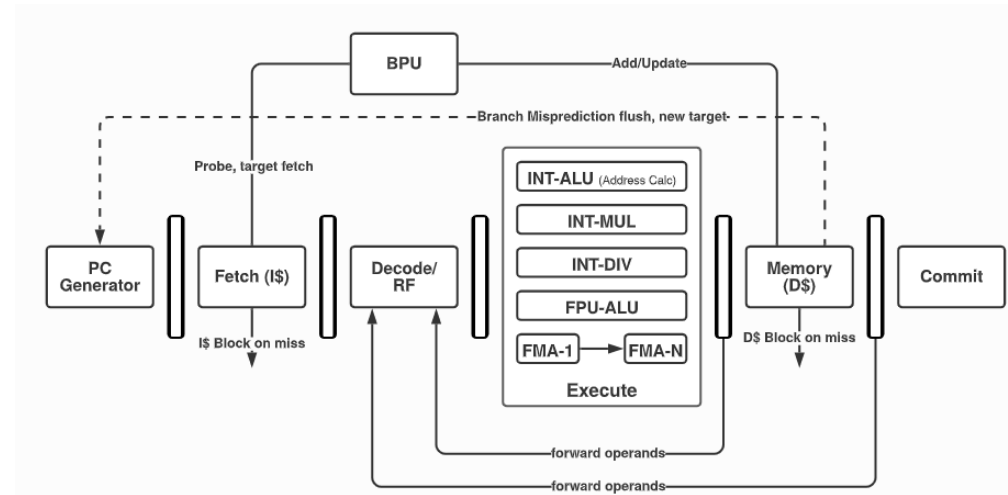


Example Microarchitectures

Intel Skylake OOO



Inorder Core





Example Instruct Set Architectures

X86 Manual

Entire RISC-V Manual



RISC-V				RISC-V Reference Card			
Base Integer Instructions (RV32I)				RV32M (RV32M)			
Category	Name	Format	Instruction	Category	Name	Format	Instruction
Loads	Load Byte	I	ld rd, rs1, imm	Stores	Store Byte	S	sb rd, rs1, imm
	Load Halfword	I	lh rd, rs1, imm		Store Halfword	S	sh rd, rs1, imm
	Load Word	I	lw rd, rs1, imm		Store Word	S	sw rd, rs1, imm
	Load Byte Unaligned	I	lb rd, rs1, imm		Store Byte Unaligned	S	sb rd, rs1, imm
	Load Halfword Unaligned	I	lh rd, rs1, imm		Store Halfword Unaligned	S	sh rd, rs1, imm
Stores	Store Byte	S	sb rd, rs1, imm	Shifts	Shift Left	S	sl rd, rs1, imm
	Store Halfword	S	sh rd, rs1, imm		Shift Right	S	sr rd, rs1, imm
	Store Word	S	sw rd, rs1, imm		Shift Left Immediate	S	sl rd, rs1, imm
	Store Byte Unaligned	S	sb rd, rs1, imm		Shift Right Immediate	S	sr rd, rs1, imm
	Store Halfword Unaligned	S	sh rd, rs1, imm		Shift Right Arithmetic	S	sr rd, rs1, imm
Arithmetic	ADD	R	add rd, rs1, rs2	Logical	XOR	R	xor rd, rs1, rs2
	ADDI	I	addi rd, rs1, imm		OR	R	or rd, rs1, rs2
	SUBTRACT	R	sub rd, rs1, rs2		AND	R	and rd, rs1, rs2
	SLT	R	slt rd, rs1, rs2		ANDI	I	andi rd, rs1, imm
	SLTI	I	slti rd, rs1, imm	Compare	SLT	R	slt rd, rs1, rs2
Logical	XOR	R	xor rd, rs1, rs2		SLTI	I	slti rd, rs1, imm
	OR	R	or rd, rs1, rs2		SLTIU	I	sltiu rd, rs1, imm
	AND	R	and rd, rs1, rs2		SLTIU	I	sltiu rd, rs1, imm
	ANDI	I	andi rd, rs1, imm		SLTIU	I	sltiu rd, rs1, imm
Compare	SLT	R	slt rd, rs1, rs2	Branches	BEQ	R	beq rd, rs1, rs2
	SLTI	I	slti rd, rs1, imm		BNE	R	bne rd, rs1, rs2
	SLTIU	I	sltiu rd, rs1, imm		BNEZ	R	bnez rd, rs1, imm
	SLTIU	I	sltiu rd, rs1, imm		BNEZ	R	bnez rd, rs1, imm
	SLTIU	I	sltiu rd, rs1, imm		BNEZ	R	bnez rd, rs1, imm
Jump & Link	JAL	J	jal rd, rs1, imm	System	System CALL	I	scall
	JALR	J	jalr rd, rs1, imm		System BREAK	I	ebreak
	JALR	J	jalr rd, rs1, imm		System BREAK	I	ebreak
	JALR	J	jalr rd, rs1, imm		System BREAK	I	ebreak
	JALR	J	jalr rd, rs1, imm		System BREAK	I	ebreak
System	System CALL	I	scall	Counter	Counter Read CYCLE	I	rdcycle
	System BREAK	I	ebreak		Counter Read CYCLE	I	rdcycle
	System BREAK	I	ebreak		Counter Read CYCLE	I	rdcycle
	System BREAK	I	ebreak		Counter Read CYCLE	I	rdcycle
	System BREAK	I	ebreak		Counter Read CYCLE	I	rdcycle



Role of a Computer Architect

- Starting from technology and demands
 - Manufacturing, e.g., 7nm.
 - Packaging, e.g., chiplet, wafer-scale, EMIB.
 - Application requirements, e.g., AI.
- Explore the design space of various components
 - In-order vs OOO
 - SRAM vs. DRAM
 - Cache vs. scratchpad
 - Coherence and consistency
- Deliver products with strong PPA metrics
 - Performance, power, area



Analogy Breakdown

We care less about a particular instance of a design...

- **Fungibility**

- No intrinsic value to a particular instance -- Don't care where my program lives

- **Durability**

- Every two years you throw away your personal out-of-order multicore chip (smartphone) and buy a new one.
- Compute devices will anyways become obsolete due to technology

- **Manufacturing Tradeoffs**

- Nth+1 chip costs $\sim \$0$



Design Goals / Constraints

- **Functional**

- Needs to be correct
 - And unlike software, difficult to update once deployed
- Security: Should provide guarantees to software

- **Reliable**

- Does it ***continue*** to perform correctly?
- Hard fault vs transient fault
- Space satellites vs desktop vs server reliability

- **High performance**

- Many factors to performance, Iron Law

- **Generality**

- “Fast” is only meaningful in the context of a set of important tasks
- Impossible goal: fastest possible design for all programs

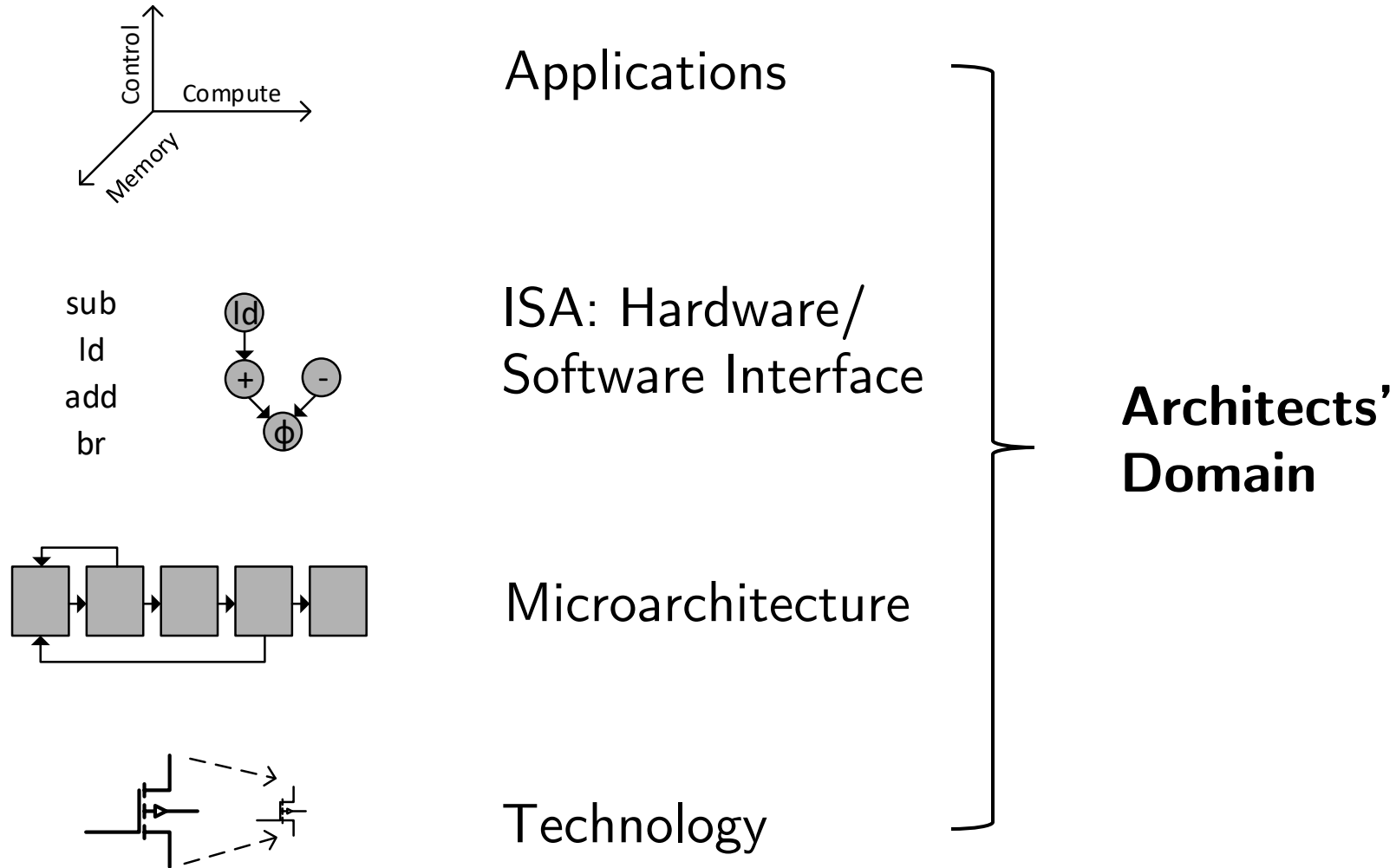


Design Goals / Constraints ... continued

- **Low cost**
 - Design/verification cost
 - Cost of making first chip after design (mask cost)
 - Per unit manufacturing cost (wafer cost)
- **Low power/energy**
 - Energy in (battery life, cost of electricity)
 - Energy out (cooling and related costs)
 - Cyclic problem, very much a problem today
- **Low carbon contribution?**
- **Challenge: balancing the relative importance of these goals**
 - And the balance is constantly changing
 - No goal is absolutely important at expense of all others



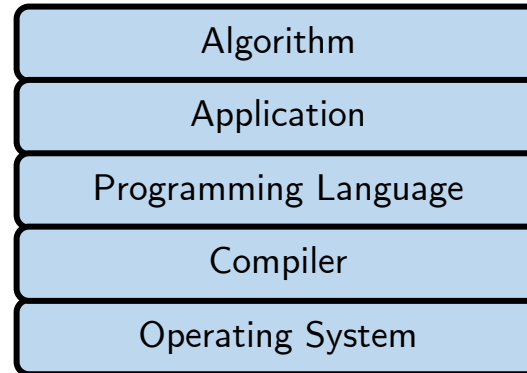
Layers of Abstraction



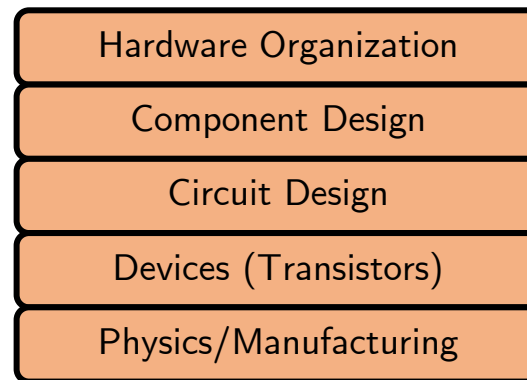


Layers of Abstraction

Software World



Instruction Set Architecture (ISA)

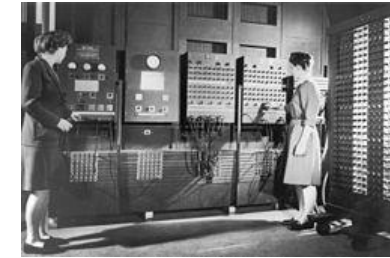


Hardware World



Early Computers: Had No “Architecture”!

- Each Computer Required Tremendous Effort
 - Hardware: Stuff that has mass
 - Software: Stuff that has no mass (instructions)
- Problem: Boundary between hardware/software was fuzzy
 - No standard interface to specify programs!
 - New programs require fiddling with hardware, yuk!
 - New hardware required fiddling with programs, gross!
 - Software Lagged Hardware
- Unimaginable today
 - Going forward: Need common interface b/t HW & SW – instruction set architecture
 - Question: Is the ISA the right way to do that?



Betty Jean Jennings and Fran Bilas

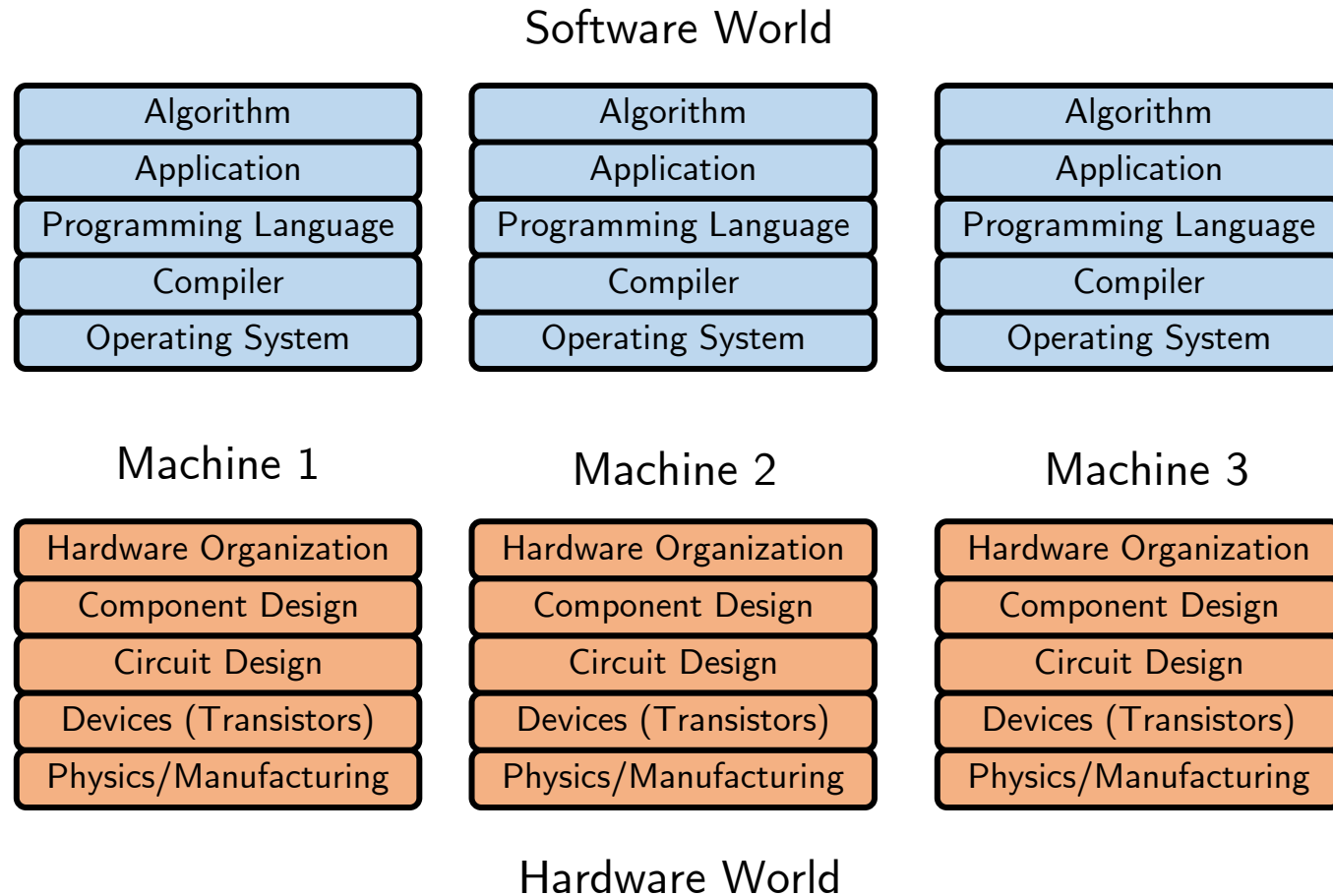


Creators: John Mauchly, J. Presper Eckert

ENIAC: First machine with abstractions, kind of
John Von Neumann -> popularized the concept of ISA

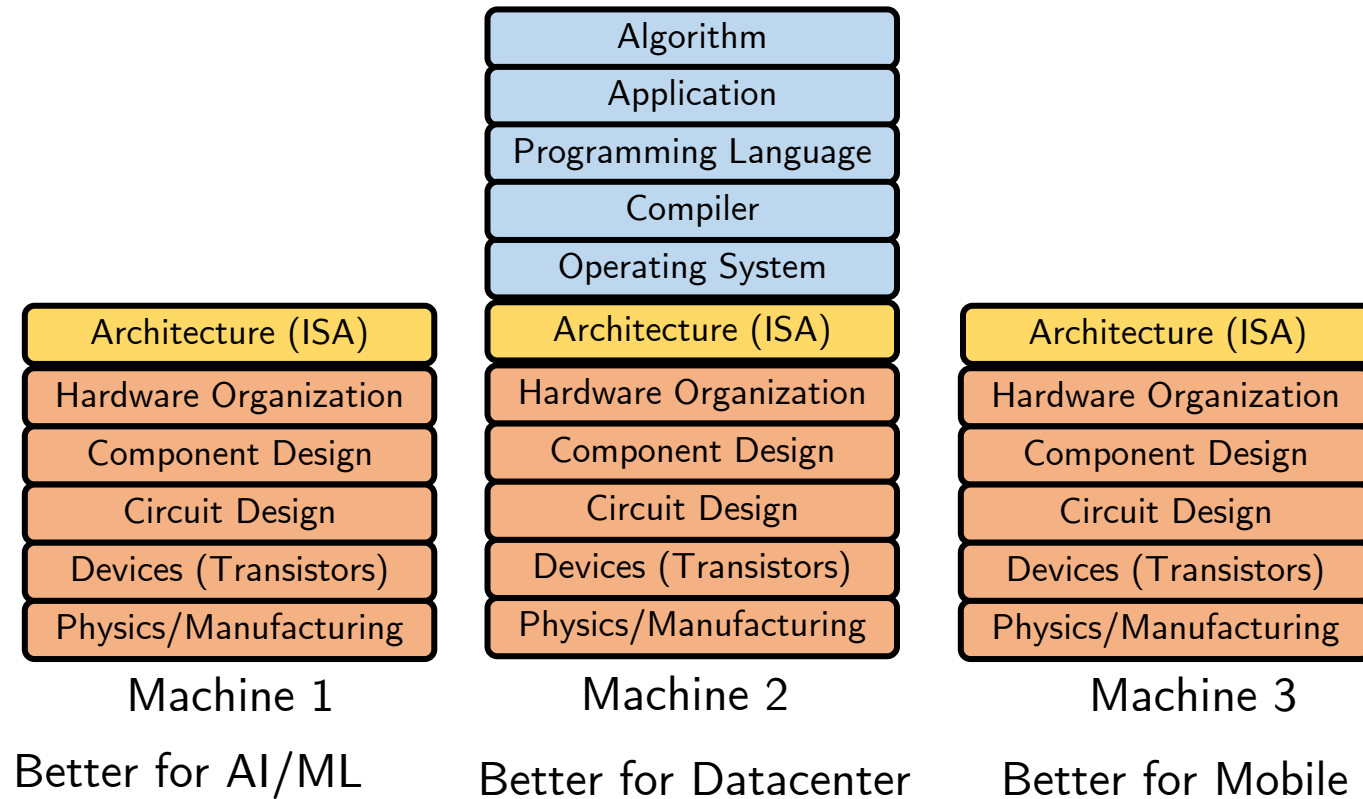


Without ISA





ISAs: Language for HW/SW Interaction





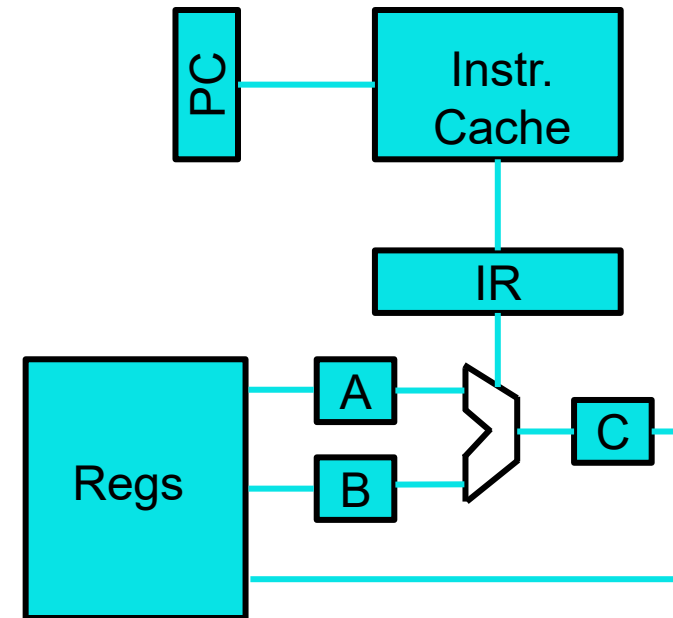
Instruction Set Architecture

- Challenge: Enable efficient expression of algorithms
- Expose information that hardware can use... (ok, vague)
- Expose Parallelism
 - Instruction Parallelism
 - Lack false serialization/dependences
 - What else?
- A good compiler target
 - Simple & orthogonal
- Dense
 - Reduce cost of reading/storing instructions
- Permits Control/Virtualization
 - Interruptible/Preemptable
 - Virtual memory



Microarchitecture

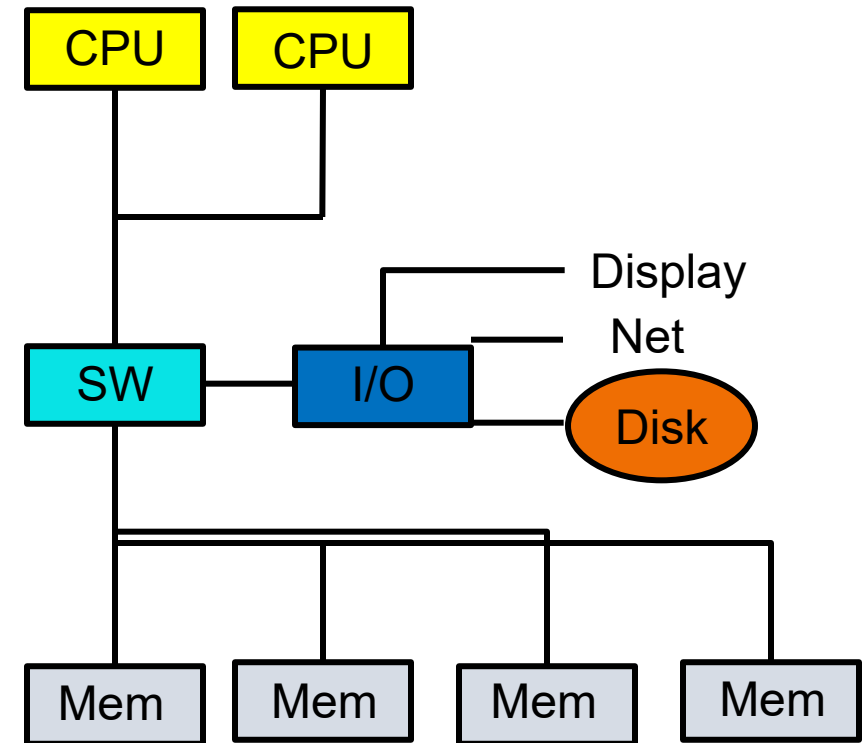
- Challenge: overcome sequential nature of programs
- Example Techniques
 - Deep pipelining
 - Multiple issue
 - Dynamic scheduling: Branch prediction/speculation
- Constraints Up-the-stack
 - Constrained by the ISA
 - Constrained by the behaviors present in applications
- Constraints Down-the-stack
 - Constrained by technology/hardware costs.
 - Physical constraints (power, area, communication)





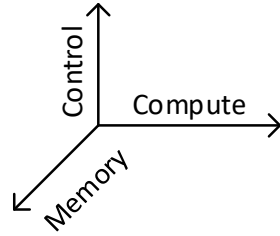
System-Level Design

- Challenge: Balancing possible bottlenecks
 - processors, memories, and interconnect
- Feeds and speeds
 - Constrained by IC pin count, module pin count, and signaling rates
- System balance
 - For a particular application
- Driven by
 - Performance/cost gains
 - Available components (cost/perf)
 - Technology constraints (I/O, network tech, manufacturing, chiplets, etc.)



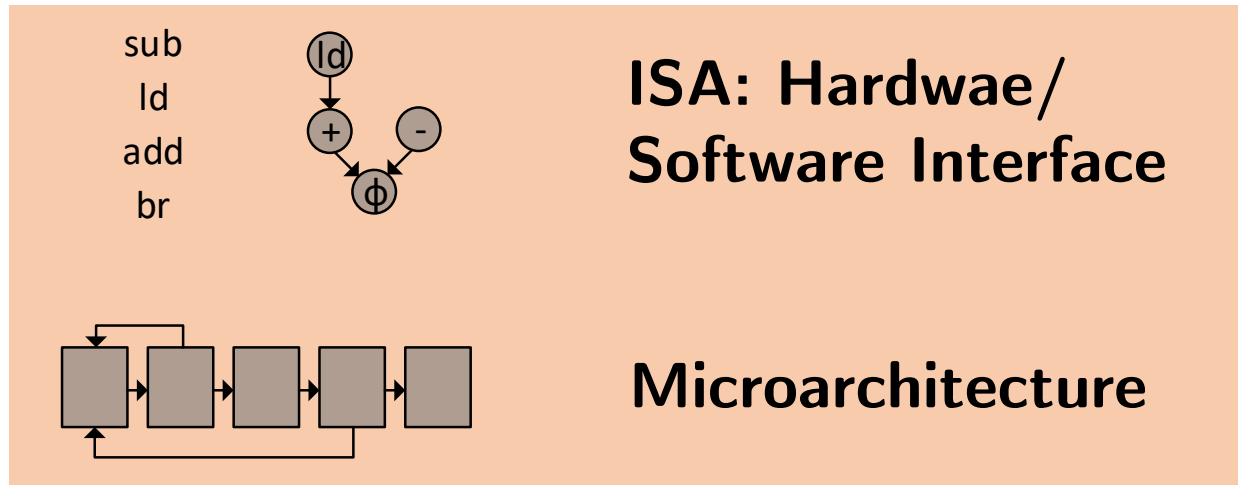


Layers of Abstraction



Applications

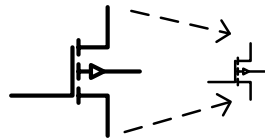
**Major Driver
Going forward?**



Architects'
Domain

Technology

**Major Driver
for 50+ years**



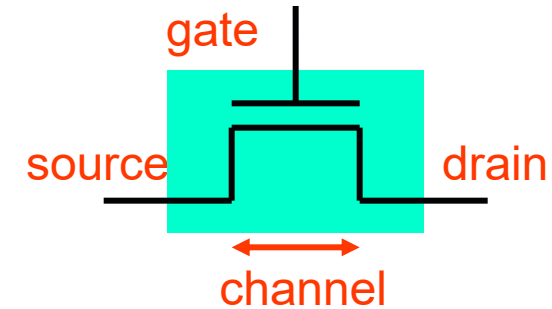


Co-evolution of Technology and Microarchitecture



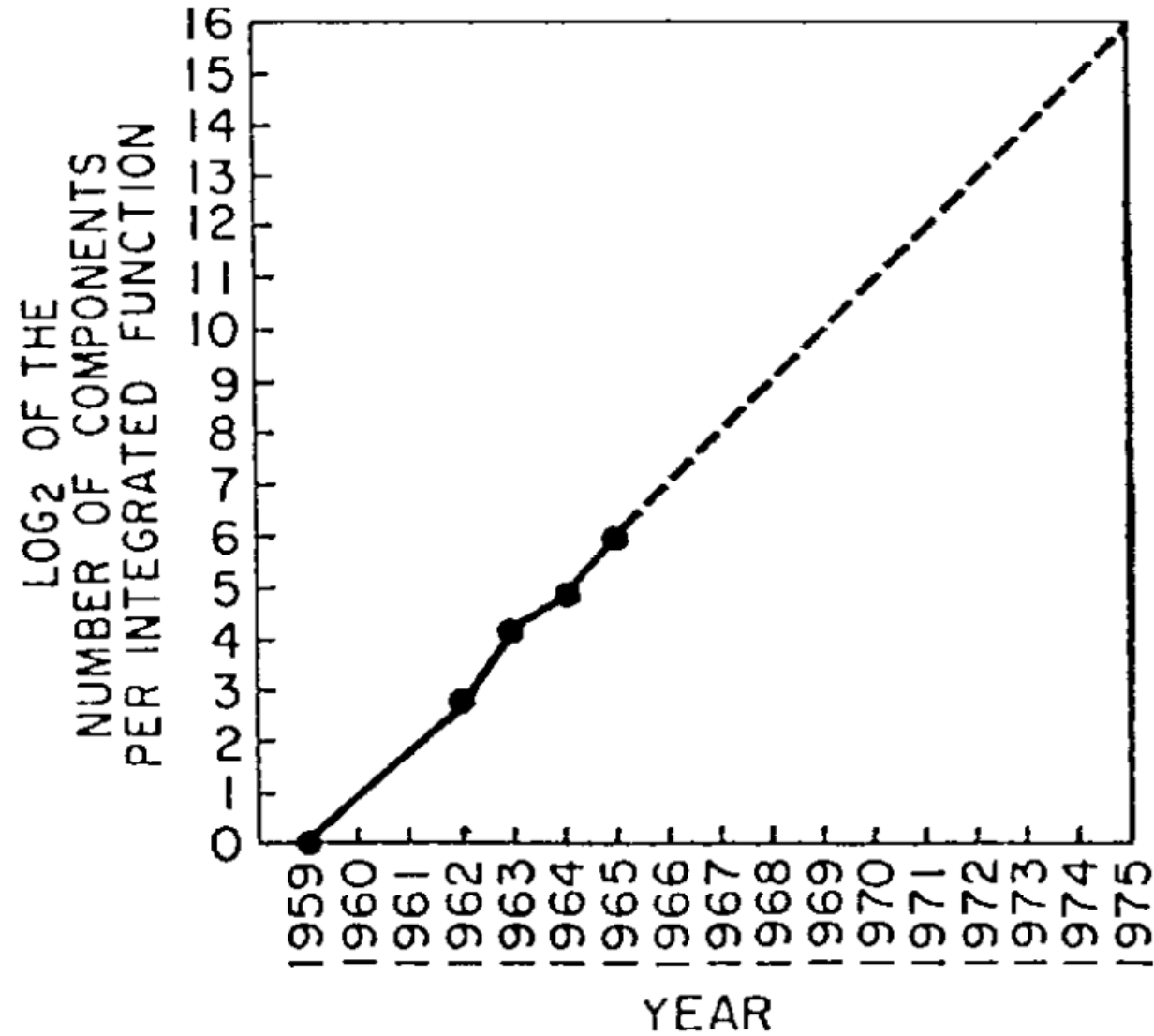
“Technology”

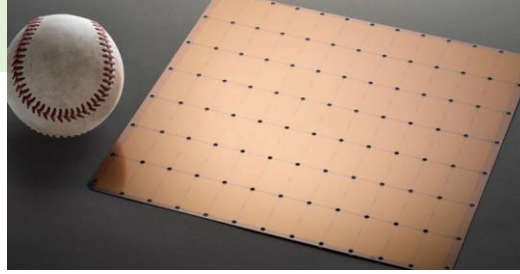
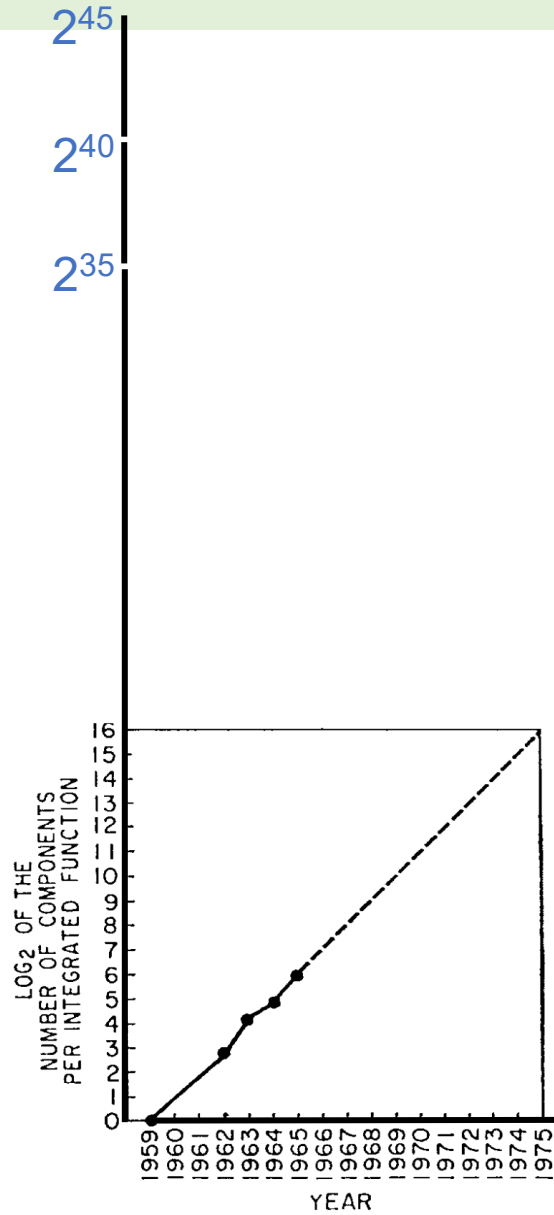
- Basic element
 - Solid-state **transistor** (i.e., electrical switch)
 - Building block of **integrated circuits (ICs)**
- Why ICs changed everything
 - High performance, high reliability, low cost, low power
 - Economies of scale via fabrication
- Several kinds of IC families
 - **SRAM/logic**: optimized for speed (used for processors)
 - **DRAM**: optimized for density, cost (used for memory)
 - **Flash/NVM**: optimized for non-volatility, density, cost (storage)
 - Increasing opportunities for integrating multiple technologies
 - Chiplets and Die Stacking
- Non-CMOS storage and interconnect technologies
 - Ethernet, fiber optics, wireless, Disks





Moore's Law -- 1965



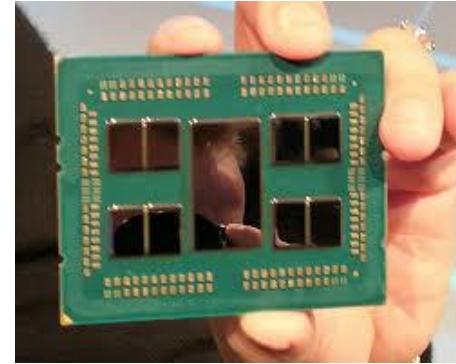


Cerebras
WS1



AMD EPYC
(chiplet)

WS3



2020

2025



Technology Change Drives Everything

- Historic/Optimistic View: Computers get 10x faster & more energy efficient every 10 years!
 - A 10x quantitative change is qualitative change
 - Plane is 10x faster than car, and fundamentally different travel mode
- New applications become self-sustaining market segments
 - Examples: laptops, mobile phones, virtual/augmented reality, autonomous vehicles, brain-computer interfaces, energy-harvesting intermittent devices etc.
- Low-level improvements appear as discrete high-level jumps
 - Capabilities cross thresholds, enabling new applications and uses



Revolution I: The Microprocessor

- **Microprocessor revolution**

- One significant technology threshold was crossed in 1970s
- Enough transistors (~25K) to put a 16-bit processor on one chip
- Huge performance advantages: fewer slow chip-crossings
- Even bigger cost advantages: one “stamped-out” component

- Microprocessors created new market segments

- Desktops, CD/DVD players, laptops, game consoles, set-top boxes, digital camera, mp3 players, GPS, mobile phones

- And replaced incumbents in existing segments

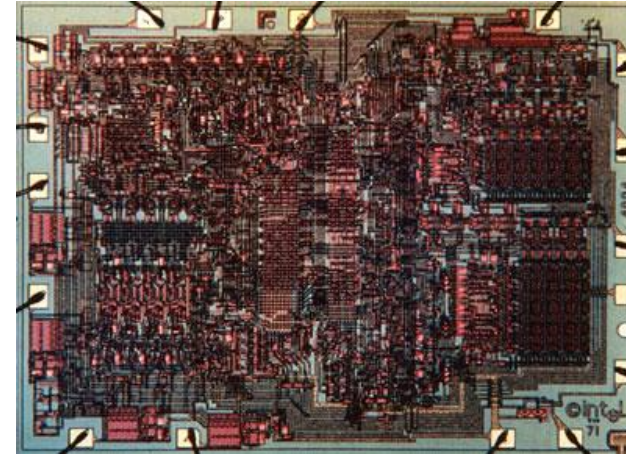
- Microprocessor-based system replaced “mainframes”, “minicomputers”, etc.



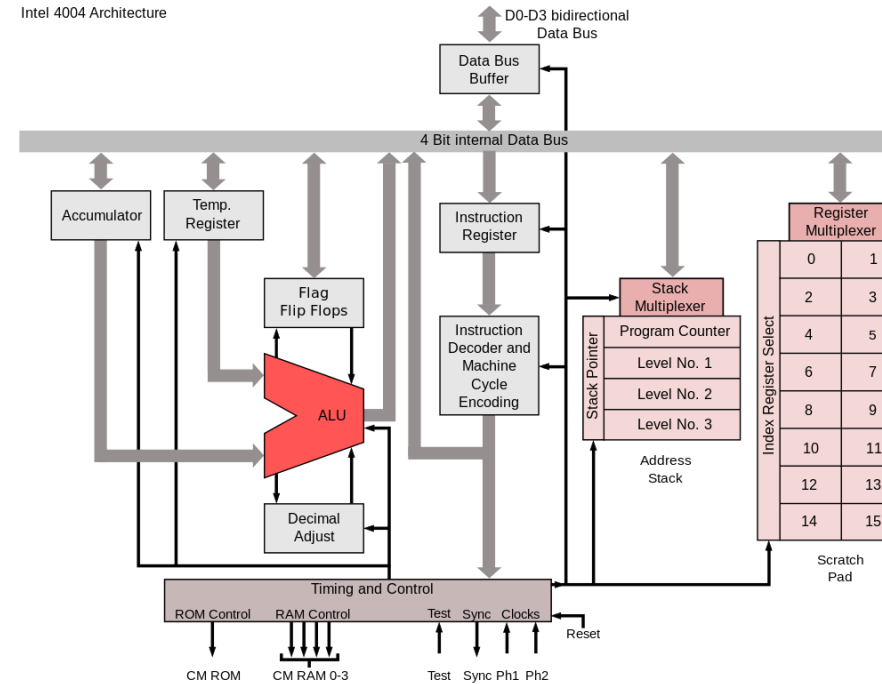


First Microprocessor

- Intel 4004 (1971)
 - Application: calculators
 - Technology: 10000 nm
 - 2300 transistors
 - 13 mm²
 - 108 KHz
 - 12 Volts
 - 4-bit data
 - Single-cycle datapath



Intel 4004 Architecture





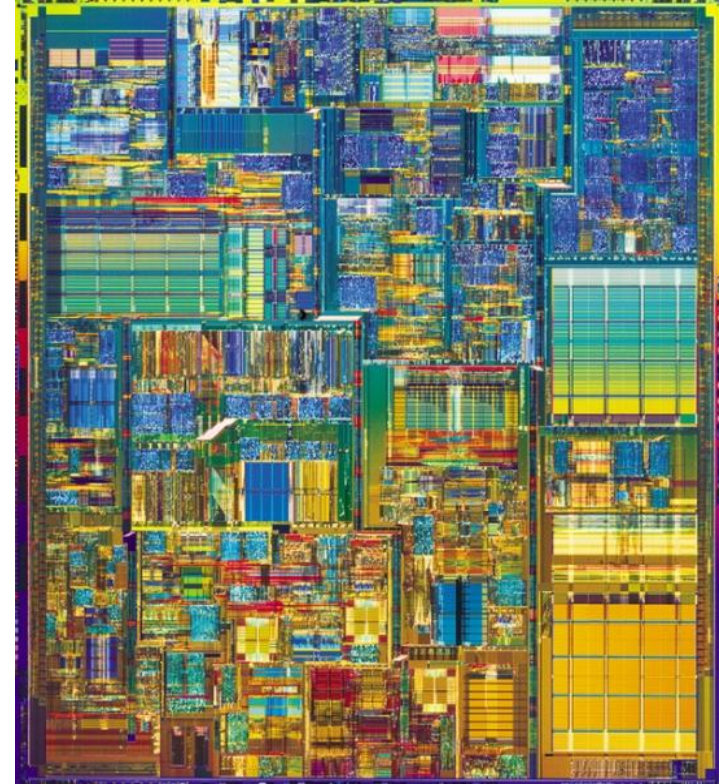
Revolution II: Implicit Parallelism

- Then to **extract implicit instruction-level parallelism**
 - Hardware provides parallel resources, figures out how to use them
 - Software is oblivious
- Initially using pipelining ...
 - Which also enabled increased clock frequency
- ... caches ...
 - Which became necessary as processor clock frequency increased
- ... and integrated floating-point
- Then deeper pipelines and branch speculation
- Then multiple instructions per cycle (superscalar)
- Then dynamic scheduling (out-of-order execution)
- We will talk about these things



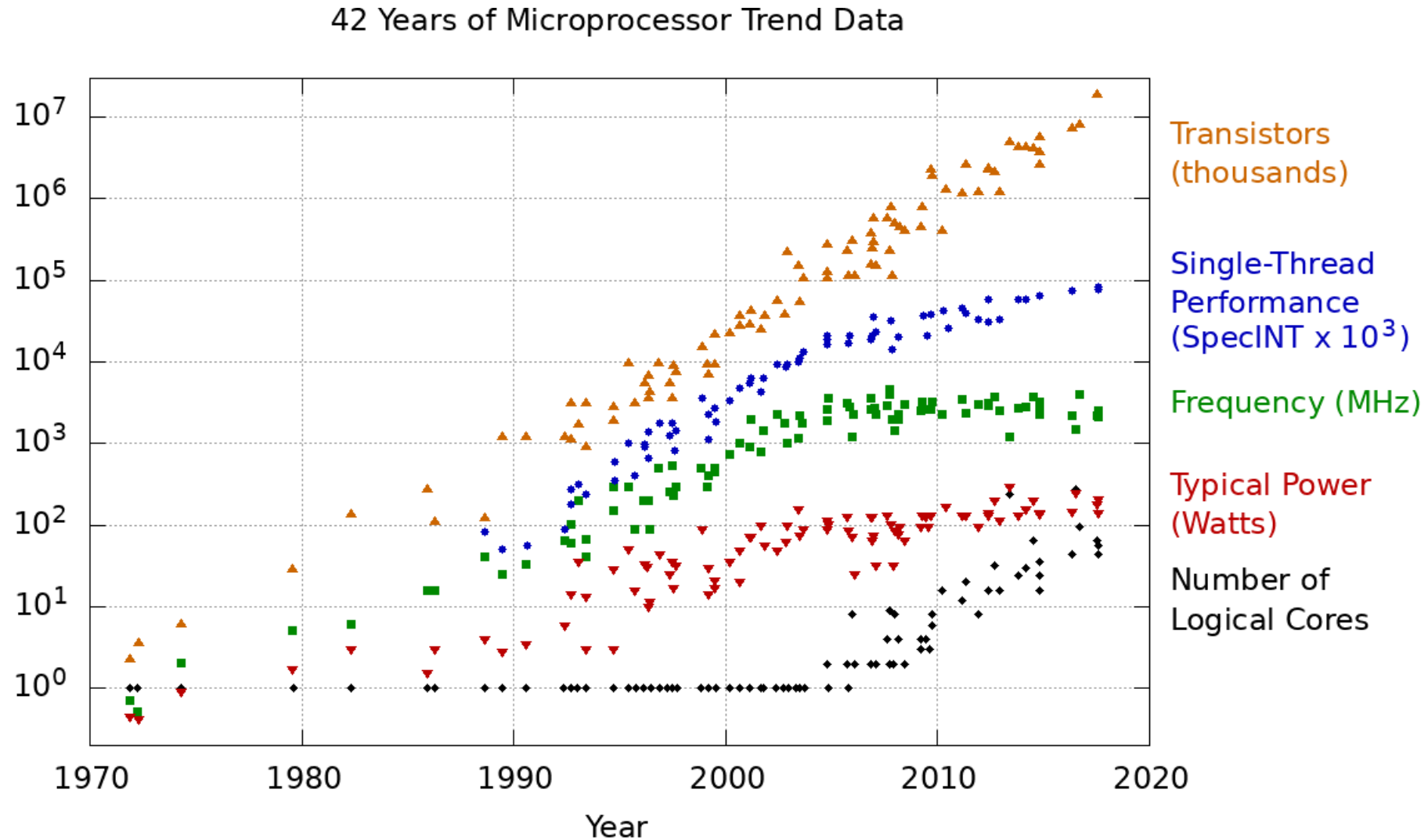
Pinnacle of single-core microprocessors

- Intel Pentium4 (2003)
 - Market: Desktop/server
 - Technology: 90nm (1/100x)
 - 55M transistors (20,000x)
 - 101 mm² (10x)
 - 3.4 GHz (10,000x)
 - 1.2 Volts (1/10x)
 - 32/64-bit data (16x)
 - 22-stage pipelined datapath (22x)
 - 3 instructions per cycle (superscalar)
 - Two levels of on-chip cache
 - data-parallel vector (SIMD) instructions, hyperthreading





Continue the Scaling



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp



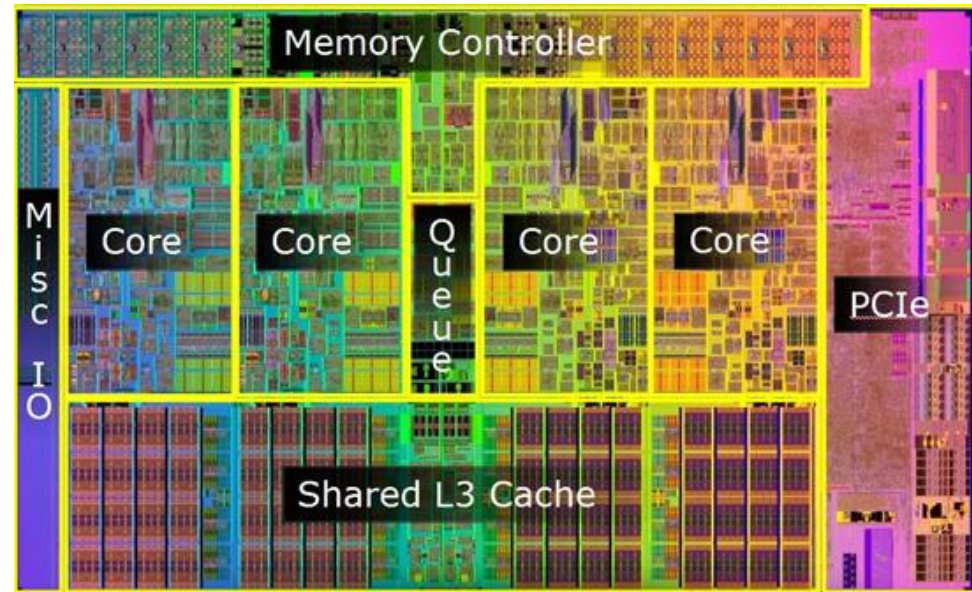
Revolution III: Explicit Parallelism

- Support **explicit data & thread level parallelism**
 - Hardware provides parallel resources, software specifies usage
 - Why? diminishing returns on instruction-level-parallelism
- First using (subword) vector instructions..., Intel's SSE
 - One instruction does four parallel multiplies
- ... and general support for multi-threaded programs
 - Coherent caches, hardware synchronization primitives
- Then using support for multiple concurrent threads on chip
 - First with single-core multi-threading, now with multi-core



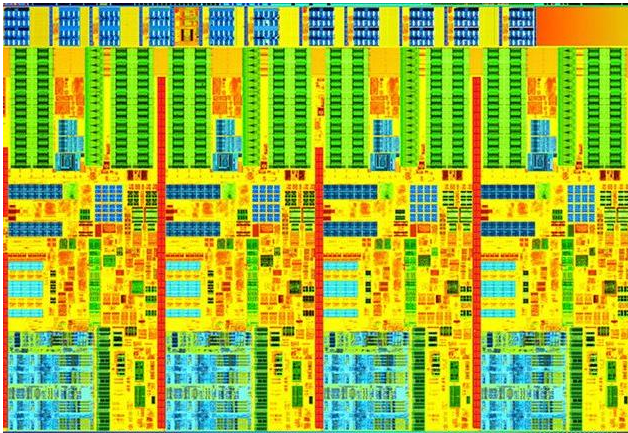
Multicore Processor

- Intel Core i7 (2009)
 - Application: desktop/server
 - Technology: 45nm (1/2x)
 - 774M transistors (12x)
 - 296 mm² (3x)
 - 3.2 GHz to 3.6 Ghz (~1x)
 - 0.7 to 1.4 Volts (~1x)
 - 128-bit data (2x)
 - 14-stage pipelined datapath (0.5x)
 - 4 instructions per cycle (~1x)
 - *Three levels of on-chip cache*
 - *data-parallel vector (SIMD) instructions, hyperthreading*
 - **Four-core multicore (4x)**

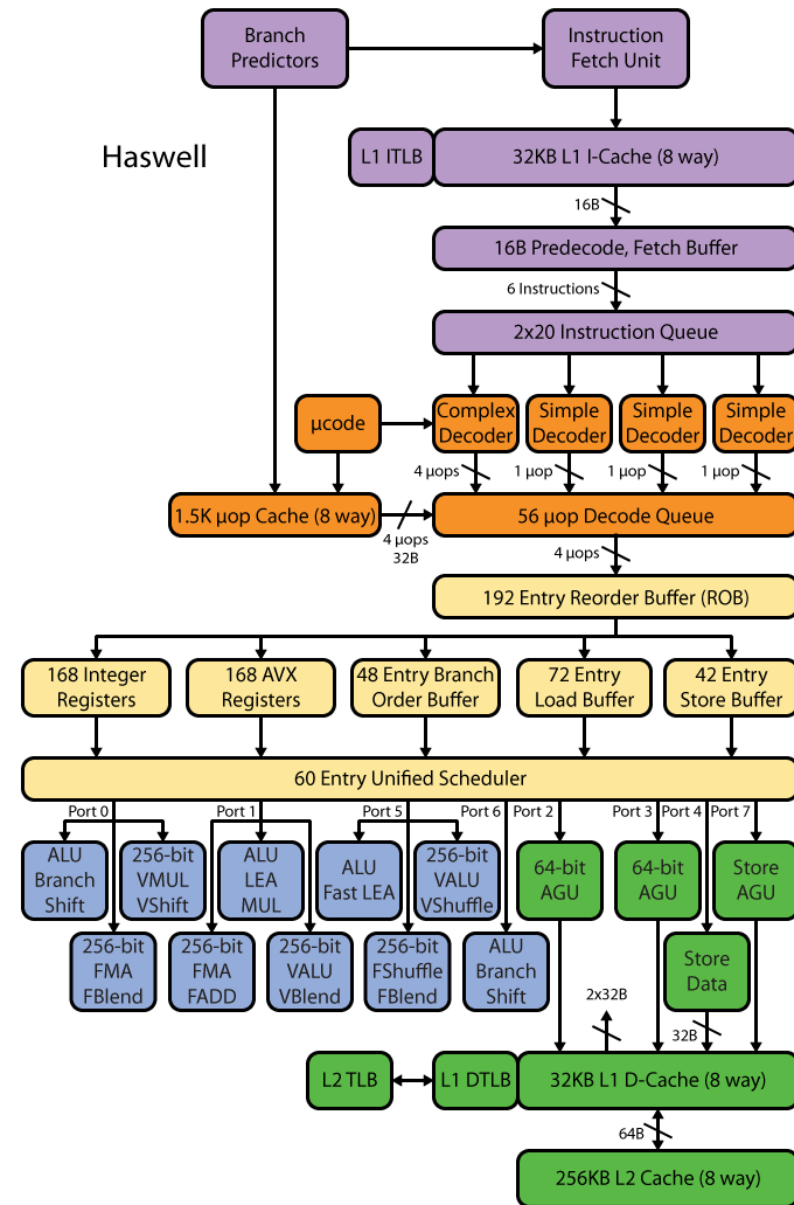




General Purpose 12 Years Ago 2014

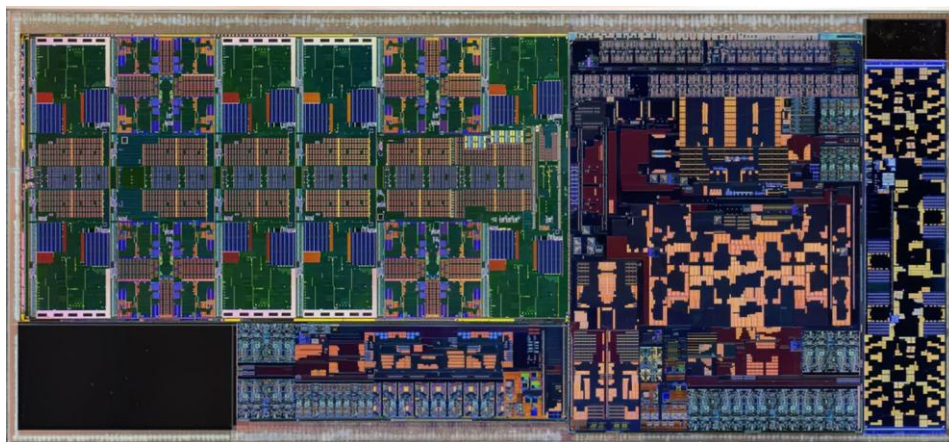


Intel Haswell





General Purpose in 2026

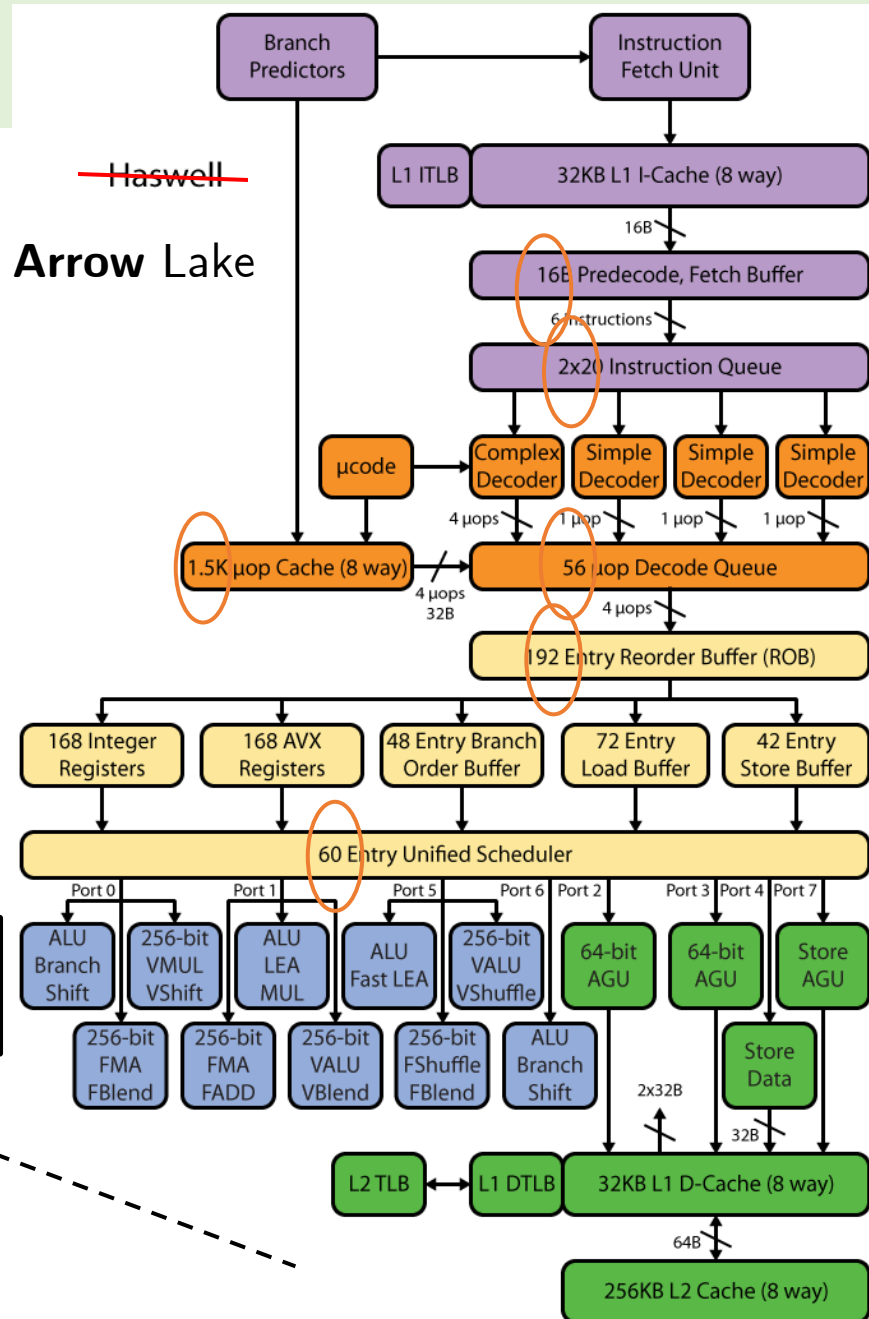


Arrow Lake

~2x Single Core Speedup

Matrix Multiply

~~Haswell~~
Arrow Lake





Oh, Intel (Well Thanks to Trump!)

- Nvidia: 5.55T (~10x)





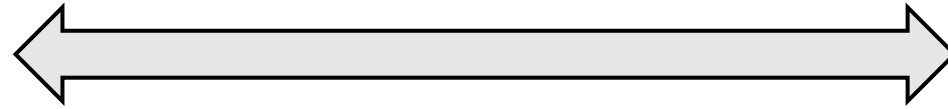
Revolution IV: Specialization

- Combine implicit/explicit parallelism with a focus on a particular domain
 - Scope can be very different: GPGPUs are quite broad, while TPUs are not...
- Tradeoff the overheads of supporting “general purpose” workloads for efficiency on a smaller set of workloads.
- But why is this happening now?
 - Because its hard and there wasn't a reason to do it before.
(tech scaling was enough to satisfy exponential growth in speed/efficiency)
 - Energy and Area are now big limiting factors



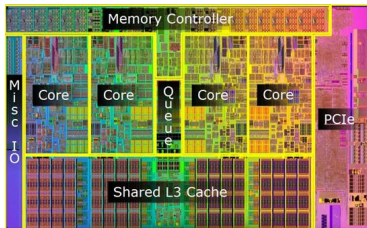
Specialization Spectrum

General Purpose

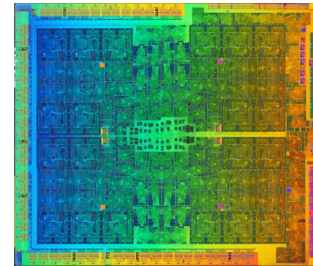


Application Specific

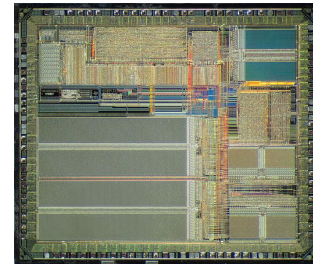
CPU
(“Ordinary” Apps)



Graphics Proc.
Unit (GPU)



Digital Signal
Proc. (DSP)



Field Prog.
Gate Array
(FPGA)

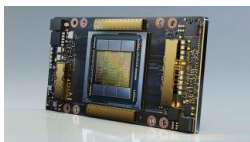


Google TPU:
(Deep Learning)





Machine learning in Industry



**NVIDIA
A100**



**Google
TPU**



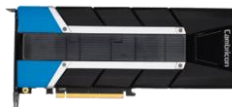
**Microsoft
Brainwave**



**Amazon
Inferentia**



**Baidu
Kunlun**



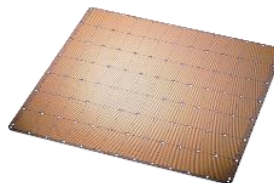
**Cambricon
MLU-100**



**GraphCore
Colossus**



**Groq
TSP**



**Cerebras
WSE**



**Habana
Gaudi**



**Mythic
IPU**



**HAILO
HAILO-8**



**Tenstorrent
Grayskull**



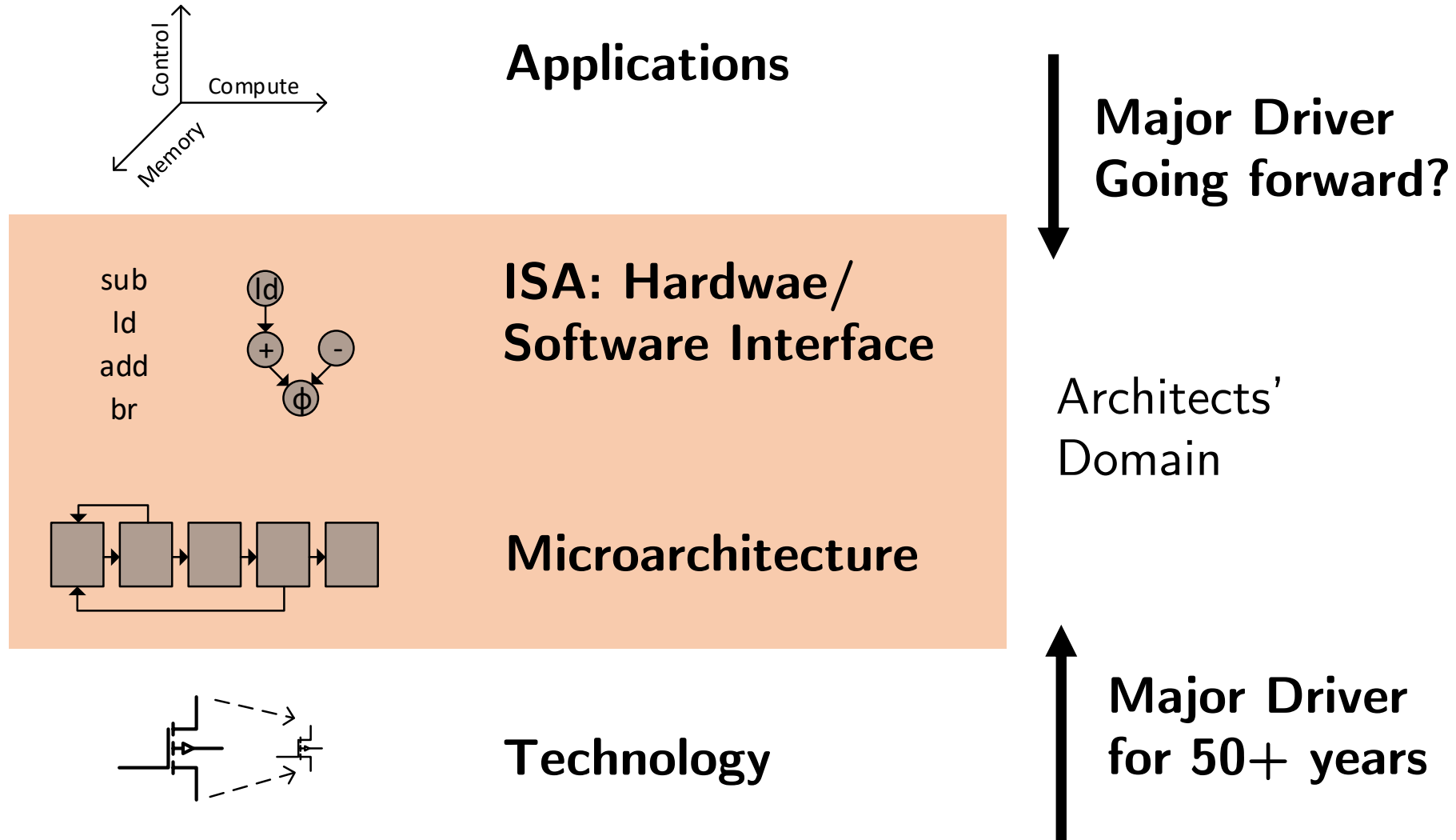
**Tesla
D1**

+Apple & Meta

+Huawei & Alibaba & many Chinese startups...



Layers of Abstraction





Applications as a Driver



Applications Views

- Many ways to view distinction between application settings:
 - **Deployment-centric view:**
 - Where the machine is deployed affects the set of applications
 - Desktop, Server, Mobile, IoT
 - **Domain-centric view:**
 - Set applications from a similar background
 - **Property-centric view:**
 - Set of applications with different properties



Deployment Centric View

- **Desktop**: home office, multimedia, games
 - Need: integer, memory bandwidth, integrated graphics/network?
- **Mobile**: laptops, mobile phones
 - Need: **low energy**, heat, integer performance
- **Embedded**: Wearables, IoT
 - Need: low power, **low cost**
- **Deeply Embedded**: disposable “smart dust” sensors
 - Need: extremely low power, extremely low cost



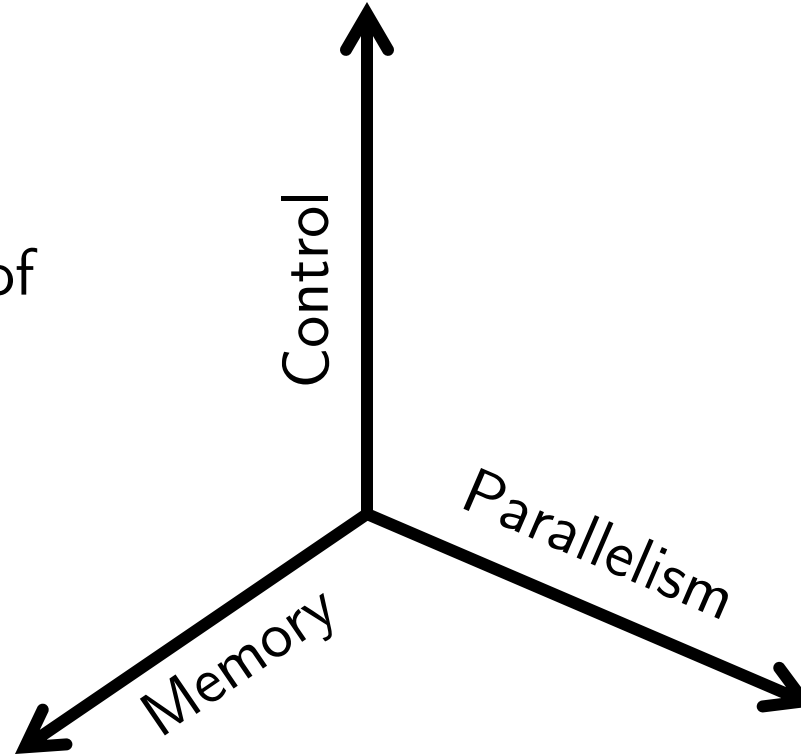
Domain-centric View

- Old Dichotomy:
 - **Scientific**: weather prediction, physical simulation, genome sequencing
 - First computing application domain: naval ballistics firing table
 - Need: large memory, heavy-duty floating point
 - Examples: CRAY T3E, IBM BlueGene
 - **Commercial**: database/web serving, e-commerce
 - Need: data movement, high memory + I/O bandwidth
 - Examples: Sun Enterprise Server, AMD Opteron, Intel Xeon, IBM Power 7
- **Recently – finer grain domains:**
 - Deep learning, Digital Signal Processing, Graphics, Genomics, Database Processing, Compression/Decompression, Molecular Dynamics (anton by DE shaw)



Property-centric View

Dimensions of
Application
“Regularity”



More regularity → Less data dependences

Less dependences → Easier to run efficiently



Control Regularity

Increasing “Irregularity”



- No Control (or non critical)
- Data-Independent
- Data-Dependent, Predictable
- Data-Dependent, Unpredictable

(also, indirect branches)

```
for i
  ... = a[i]
```

```
for i
  if(i%2)
    ... = a[i]
```

```
for i
  if(age[i]>2)
    ... = a[i]
```

```
for i
  if(age[i]>22)
    ... = a[i]
```




Memory Regularity

Increasing “Irregularity”

- Data dependence

```
for i=0 to n  
  ... = a[i]
```

```
while(node)  
  ... = *node  
  node = node->next
```

- Alias freedom

```
for i=0 to n  
  ... = a[i]
```

```
for i=0 to n ... =  
  a[index[i]]++
```



Parallelism Regularity

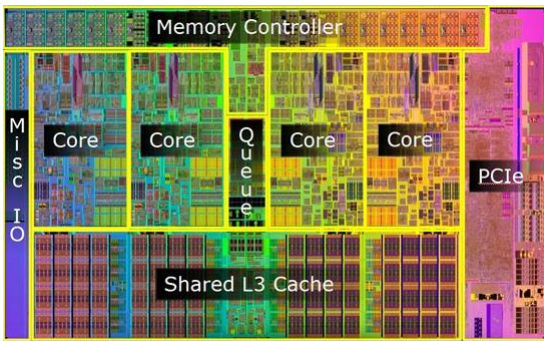
- Types of Parallelism
 - Instruction-level Parallelism (ILP): Nearby instructions running together
 - Thread-level (task-level) Parallelism (TLP): Independent threads (at least to some extent) running simultaneously.
 - Data-level Parallelism (DLP): Do same thing to many pieces of data.
- Which is the most regular: (one perspective)
 - DLP: more regular
 - TLP: less regular (dependences infrequent)
 - ILP: least regular (more frequent dependences)
- Dimensions of Regularity
 - Granularity: Fine vs Coarse Grain
 - Data-dependence: Static vs Dynamic
- Complexity Ahead:
 - DLP implies ILP... (but not other way around)
 - DLP implies TLP ... (but not other way around)



A naïve property-based classification

CPU

("Ordinary" Apps)

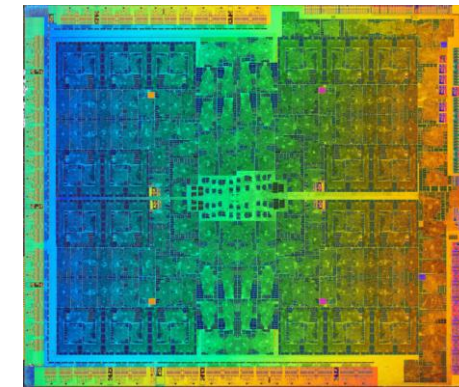


- + Irregular Control/Memory
- + Fine-grain Instruction Level Parallelism

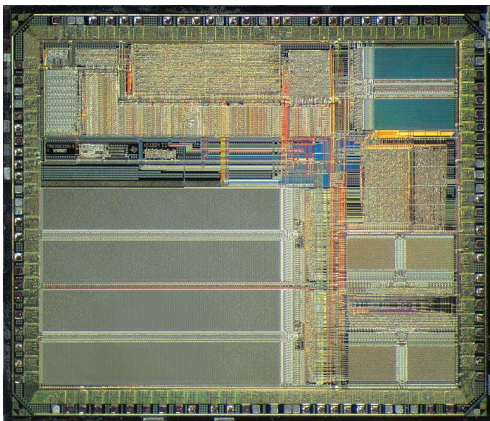
- + Thread-level and Data-level Parallelism
- + Medium Control/Memory

GPU

(Graphics, Data-proc.)



DSP (signal proc.)



- + Fine-grain ILP
- + Highly-regular Memory

- + Extremely Regular Data Parallelism
- + Extreme Control/Memory Regularity

Google TPU:

(Deep Learning)



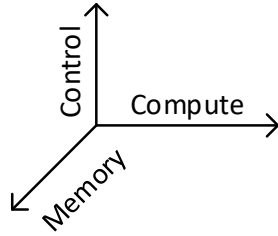


Other properties

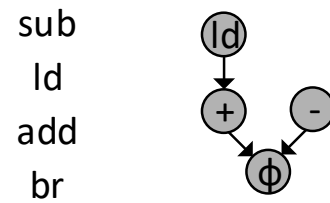
- Locality
 - Temporal:
 - Do instructions/data repeat within some time?
 - Loopy vs function-call code
 - Spatial:
 - Reuse related data/instructions in sequence?
- Data and instruction types
 - Integer vs Floating Point
 - Small vs Large Bitwidth
 - (Ratio of resources + conversion overheads)
- Probably many other important properties, depending on the context and architecture...



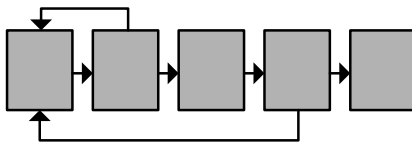
Course Theme: Thinking Across Layers



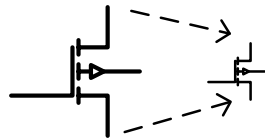
Applications



ISA: Hardware/ Software Interface



Microarchitecture



Technology



CENG 5410 Course Overview



Some Course Goals

- Exposure to “big ideas” in computer architecture
 - Pipelining, parallelism, caching, locality, abstraction, etc.
 - Understand the “why” behind architecture/systems tradeoffs
 - Exposure to examples of good (and some bad) engineering
- “Practical” Skills
 - Simulators/C++
 - Empirical evaluation/analysis/experiment design
 - Programming for performance
- Research exposure (through papers/project)
 - Evaluating your own idea is rewarding!
 - Skepticism without Cynicism
- Understand where computers are going
 - Future capabilities drive the computing world
 - Forced to think 5+ years into the future



Course Prerequisites

- Basic Computer Organization
 - Logic: gates, Boolean functions, latches, memories
 - Datapath: ALU, register file, muxes
 - Control: single-cycle control, micro-code
 - Caches & pipelining (will go into these in more detail here)
 - Some familiarity with assembly language
 - Hennessy & Patterson's "Computer Organization and Design"
 - Operating Systems (processes, threads, & virtual memory)
- Significant programming experience
 - Why? assignments require writing code to simulate hardware
 - Not difficult if competent programmer; extremely difficult if not
- **This class will have a gentle ramp, so don't worry.**



Course Components

- Discussions:
 - We will read research papers from literature, including classic and modern works. Participate in discussion on Blackboard one or two papers each week.
- Midterm Exam:
 - One in-class midterm.
 - There will be no final exam.
- Project:
 - Option 1: Hacking the gem5/FlashGPU-sim simulator based on a suggested idea (evaluate new/existing architecture idea using simulation).
 - Option 2: Open ended project of your choice.



Paper Readings/Reviews

- You should participate in class if possible.
 - Bonus points for participation (up to 5% of class grade)
- Online Discussion
 - Meaningfully participate in the discussion of 8 papers during the quarter.
 - Post early and build on other students' participation (most post before corresponding class starts)
- What is meaningful participation (in order):
 - **Novel Idea / Augmentation** – Original idea, no need to be good
 - **Non-trivial Question** (only if you are 24 hours before deadline)
 - **Critique** -- a significant flaw/shortcoming (no methodology)
 - **Impact** -- Impact beyond stated contributions
- Reserve right to take off for redundant answers. :)
- Please do not use AI entirely! Produce your own idea/opinion/logic.
 - That's where the fun is :)



Required Texts

- No required *traditional* textbook for this course.
- Required reading will include:
 - Springer Synthesis Lectures – link on website
 - Published papers (available on campus network through ACM and IEEE libraries).
- Optional Textbooks:
 - John Shen and Mikko Lipasti, Modern Processor Design: Fundamentals of Superscalar Processors, McGraw-Hill, 2005.
 - John L. Hennessy and David A. Patterson, Computer Architecture: A Quantitative Approach Morgan Kaufmann Publishers, Sixth Edition.



Midterm Exam

- One exam (no final), testing 2/3 of material each
- Exam Format: In-class exam. Open book, but no internet.
- Exam Content:
 - 3-5 short analysis questions (focusing on concepts)
 - Topic fair game: anything discussed in class or in readings
 - Questions may be about a new aspect of a relevant subject



Project

- In lieu of final exam, we will have a ~8 week project.
 - Start before the midterm exam!
 - Work in teams of 2-3. (preferably not 1 or 4...)
- Why Project?
 - Give you a chance to put into practice some of the ideas
 - Give you freedom to work on something you like
 - Learn tools/approach that can be useful later
- What is a project? Options:
 - Hacksim: Hack an architecture simulator to implement a new architecture or microarchitecture idea, and evaluate it.
 - Open-ended: Propose a research idea and evaluate it using any means (okay to combine with ongoing/concurrent work)
- Deliverables: report (+ source code if applicable)
 - Report should be similar to research papers (but shorter)
 - Bonus if code merged into upstream code base.



Grading Scheme

- Reviews: 30%
- Midterm exam: 30%
- Final project: 40%

Grading philosophy for graduate courses:

- Grade is only an incentive to meet minimal requirements
- Intended curve of the class is from B+ to A.
 - B+ means you tried on everything, but didn't do that well on the exams or project
- Your own motivation will determine whether you get anything meaningful out of this course.



Logistics

- Blackboard:
 - Turning things in and reporting grades.
 - Also used for review discussion (easier for us to grade than piazza).
- Webpage: <https://seanzw.github.io/courses/ceng-5410/2026-fall/>
 - Post lectures, course schedule, project description, etc...
 - Lecture PDFs are on the course schedule, won't be 100% up to date until just before class.
- Piazza: (<https://piazza.com/cuhk.edu.hk/fall2026/ceng5410>)
 - This link is on the course webpage
 - Discussions & announcements



Contact Me & TA

- Email:
 - zhengrongwang@cuhk.edu.hk
 - Put [CENG5410] in subject line
- Office Hours: SHB931,
 - Wednesday (3-5pm)
- TA: Xingyu Jiang
 - xyjiang26@cse.cuhk.edu.hk
- TA: Yifan Guo
 - yfguo26@cse.cuhk.edu.hk